Prinsip SOLID Bagian 1 — SRP, OCP, dan LSP

Mata Kuliah:
CIF61123 – Pola Perancangan (3 SKS)
Pekan ke-4 | Sub-CPMK13
Disusun oleh:
Kelompok Pengelola Mata Kuliah Pola Perancangan



Versi Dokumen: 1.0
Tanggal: 17; Bulan: Agustus; Tahun: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

Peta Perkuliahan

Kedudukan pertemuan ini di antara enam belas pekan perkuliahan

01	Konsep dasar & klasifikasi pola
02	Prinsip perancangan berorientasi objek
03	Interface, coupling–cohesion, UML
04	**SOLID bagian 1 — SRP, OCP, LSP**
05	SOLID bagian 2 — ISP, DIP
06	Code smell & refactoring
07	Klaster 1 — kontrol pembuatan objek
08	Ujian Tengah Semester
09	Klaster 2 — komposisi rekursif
10	Klaster 3 — pembungkus antarmuka
11	Klaster 4 — algoritma bervariasi
12	Klaster 5 — ketergantungan longgar
13	Katalog GoF & pedoman pemilihan
14	Studi kasus terpadu bagian 1
15	Studi kasus terpadu bagian 2
16	Ujian Akhir Semester

© Fakultas Ilmu Komputer, Universitas Brawijaya

SubCPMK yang Didukung

Pekan 4 sampai 6 seluruhnya melayani Sub-CPMK13

13 Sub-CPMK13

Mampu menganalisis penerapan prinsip SOLID dan melakukan refactoring pada kode program Java.

Bobot pada RPS: 15,00% — yang terbesar di antara sepuluh Sub-CPMK.

F Bentuk penilaian pekan ini

Formatif — latihan analisis kode dan diskusi di kelas.

Bobot: 0%.

Diukur secara sumatif melalui LK02, KUIS1, dan UTS.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Materi Pembelajaran

Tiga prinsip pertama; ISP dan DIP dibahas pekan depan

1 Mengenal SOLID

Asal-usul, tujuan, dan kedudukannya terhadap pola perancangan.

2 Single Responsibility

Satu kelas, satu alasan untuk berubah.

3 Open/Closed

Terbuka bagi perluasan, tertutup bagi modifikasi.

4 Liskov Substitution

Turunan harus dapat menggantikan induknya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tujuan Pembelajaran

Setelah pertemuan ini mahasiswa diharapkan mampu:

- Menjelaskan kedudukan prinsip SOLID sebagai landasan penilaian mutu rancangan berorientasi objek.
- Menjelaskan dan menerapkan Single Responsibility Principle beserta perumusannya menurut aktor.
- Menjelaskan dan menerapkan Open/Closed Principle melalui abstraksi dan polimorfisme.
- Menjelaskan dan menerapkan Liskov Substitution Principle serta mengenali pelanggaran kontrak perilaku.
- Mengidentifikasi pelanggaran ketiga prinsip pada potongan kode program Java dan mengusulkan perbaikannya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

01

MATERI KE-1

Mengenal Prinsip SOLID

Bagian 1 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

SOLID — Lima Prinsip Perancangan Berorientasi Objek

Dirumuskan Robert C. Martin; akronimnya disusun Michael Feathers

S — Single Responsibility

Sebuah kelas hanya boleh memiliki satu alasan untuk berubah.

O — Open/Closed

Terbuka bagi perluasan, tertutup bagi modifikasi.

L — Liskov Substitution

Turunan harus dapat menggantikan tipe dasarnya.

I — Interface Segregation

Klien tidak boleh dipaksa bergantung pada metode yang tidak dipakainya.

D — Dependency Inversion

Bergantunglah pada abstraksi, bukan pada rincian.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kedudukan SOLID dalam Mata Kuliah Ini

Prinsip lebih dahulu, pola menyusul sebagai penerapan yang telah dibakukan

Apa yang diberikan SOLID

- **Ukuran mutu.** Rancangan dapat dinilai baik atau buruk dengan alasan yang dapat diperiksa.
- **Bahasa bersama.** Diskusi rancangan tidak berhenti pada selera.
- **Petunjuk arah.** Ketika sebuah prinsip dilanggar, biasanya sudah ada pola yang menjawabnya.

Hubungannya dengan pola GoF

- **Strategy.** Cara baku memenuhi Open/Closed pada variasi algoritme.
- **Abstract Factory.** Cara baku memenuhi Dependency Inversion pada pembuatan objek.
- **Decorator.** Cara baku menambah kemampuan tanpa mengubah kelas yang ada.
- **Adapter.** Cara baku menyelaraskan antarmuka tanpa melanggar Interface Segregation.

◆ **Intinya: pola perancangan adalah jawaban baku atas pelanggaran prinsip yang berulang.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

02

MATERI KE-2

Single Responsibility Principle

Bagian 2 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Dua Perumusan yang Saling Melengkapi

Perumusan kedua muncul pada Clean Architecture untuk menutup celah tafsir yang pertama

Perumusan klasik

- “A class should have only one reason to change.”
- **Alasan berubah, bukan fungsi.** Kata “tanggung jawab” terlalu kabur; “alasan berubah” dapat diperiksa.
- **Akibat.** Bila sebuah kelas berubah karena dua permintaan yang berbeda, ia melanggar SRP.

Perumusan menurut aktor

- “A module should be responsible to one, and only one, actor.”
- **Aktor.** Kelompok pemangku kepentingan yang memicu permintaan perubahan.
- **Akibat.** Metode yang melayani aktor berbeda harus dipisahkan ke kelas yang berbeda.

Intinya: SRP adalah cohesion yang diukur menurut aktor, bukan menurut jumlah metode.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Salah Kaprah yang Umum

Perbedaan tafsir ini menentukan benar atau salahnya jawaban pada kuis dan ujian

Tafsir yang keliru

- “SRP berarti satu kelas hanya boleh punya satu metode.”
- **Terlalu sempit.** Tafsir ini melahirkan ratusan kelas mungil tanpa manfaat.
- **Contoh.** LayananPengguna boleh memiliki daftar(), autentikasi(), dan aturSandi().
- **Alasannya.** Ketiganya melayani aktor yang sama, yaitu pengelola akun pengguna.

Tafsir yang benar

- “SRP berarti satu kelas hanya melayani satu aktor.”
- **Ukurannya jelas.** Tanyakan siapa yang akan meminta perubahan pada metode ini.
- **Contoh pelanggaran.** Satu kelas yang menghitung gaji, menyusun laporan jam, dan menyimpan data.
- **Alasannya.** Ketiganya dipicu oleh bagian keuangan, kepegawaian, dan pengelola basis data.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kasus: Kelas KrsManager pada Sistem Akademik

Satu kelas, tiga aktor, tiga alasan untuk berubah

KrsManager.java

```
public class KrsManager {

    // aktor 1 — Bagian Akademik: aturan pengambilan mata kuliah
    public boolean validasi(Krs krs) {
        return krs.totalSks() <=
batasSksMenurutIpk(krs.mahasiswa());
    }

    // aktor 2 — Pengelola Basis Data: cara dan tempat penyimpanan
    public void simpan(Krs krs) {
        Connection c = DriverManager.getConnection(URL, USER,
PASS);
        c.prepareStatement("INSERT INTO krs ...").executeUpdate();
    }

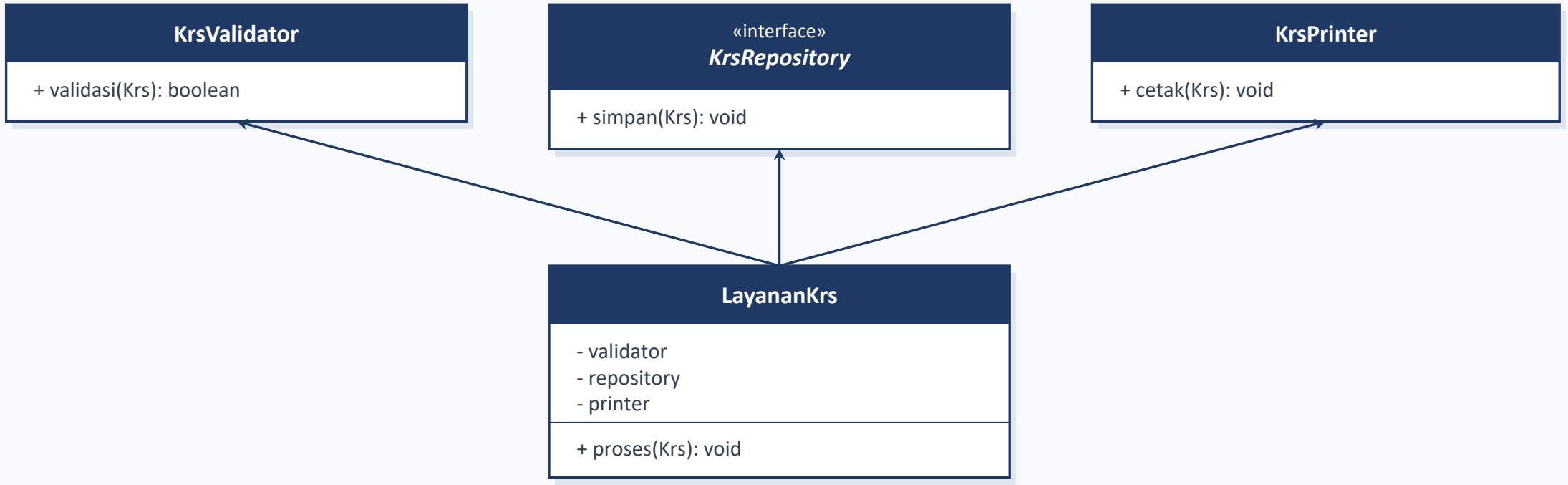
    // aktor 3 — Dosen Pembimbing Akademik: format lembar bimbingan
    public void cetak(Krs krs) {
        System.out.println("=== KARTU RENCANA STUDI ===");
    }
}
```

- **Perubahan format cetak memaksa kelas ini dikompilasi ulang bersama aturan akademiknya.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Perbaikan: Satu Aktor, Satu Kelas

Tiga alasan berubah menjadi tiga kelas; satu kelas perakit menyatukannya



- **Aturan akademik berubah — hanya KrsValidator yang tersentuh.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Cara Menguji SRP pada Sebuah Kelas

Empat langkah yang dipakai pada latihan, kuis, dan ujian

1 Daftar metodenya

Tulis seluruh metode publik kelas tersebut.

2 Tanyakan aktornya

Untuk tiap metode, siapa yang akan meminta metode itu berubah?

3 Kelompokkan

Metode dengan aktor yang sama berada dalam satu kelompok.

4 Putuskan

Lebih dari satu kelompok berarti kelas itu perlu dipecah.

© Fakultas Ilmu Komputer, Universitas Brawijaya

03

MATERI KE-3

Open/Closed Principle

Bagian 3 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Terbuka bagi Perluasan, Tertutup bagi Modifikasi

Prinsip ini mustahil dipenuhi tanpa abstraksi dan polimorfisme

Perumusan

- “Software entities should be open for extension, but closed for modification.” (Meyer, 1988)
- **Terbuka.** Perilaku baru dapat ditambahkan ketika kebutuhan bertambah.
- **Tertutup.** Kode yang sudah teruji dan berjalan stabil tidak perlu disentuh lagi.

Mengapa penting

- **Perubahan itu pasti.** Kebutuhan bisnis selalu bertambah sepanjang usia perangkat lunak.
- **Menekan risiko.** Menyentuh kode lama berarti menguji ulang seluruh perilaku lama.
- **Inti stabil, tepi lentur.** Bagian yang sering berubah didorong ke luar melalui abstraksi.

Intinya: OCP tidak berarti kode tidak pernah diubah, melainkan penambahan fitur tidak menuntut perubahan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Gejala Pelanggaran OCP

Empat tanda yang dapat dikenali tanpa menjalankan program

1 Percabangan yang panjang

Rangkaian if–else atau switch untuk memilih variasi perilaku.

2 Kata kunci instanceof

Kode memeriksa jenis objek sebelum memutuskan apa yang dilakukan.

3 Fitur baru menyentuh kode lama

Setiap penambahan menuntut berkas yang sudah stabil ikut diubah.

4 Perubahan menular

Satu penambahan memaksa beberapa berkas berbeda ikut disesuaikan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kasus: Pemilihan Metode Pembayaran

Perhatikan apa yang terjadi ketika satu metode pembayaran baru ditambahkan

Melanggar

```
class LayananPembayaran {
    void bayar(String jenis, double n) {
        if (jenis.equals("transfer")) {
            prosesTransfer(n);
        } else if (jenis.equals("kartu")) {
            prosesKartu(n);
        } else if (jenis.equals("virtual")) {
            prosesVirtual(n);
        }
    }
}

// Tambah e-wallet -> kelas ini diubah lagi.
```

Memenuhi

```
interface MetodePembayaran {
    void bayar(double jumlah);
}

class LayananPembayaran {
    private final MetodePembayaran metode;

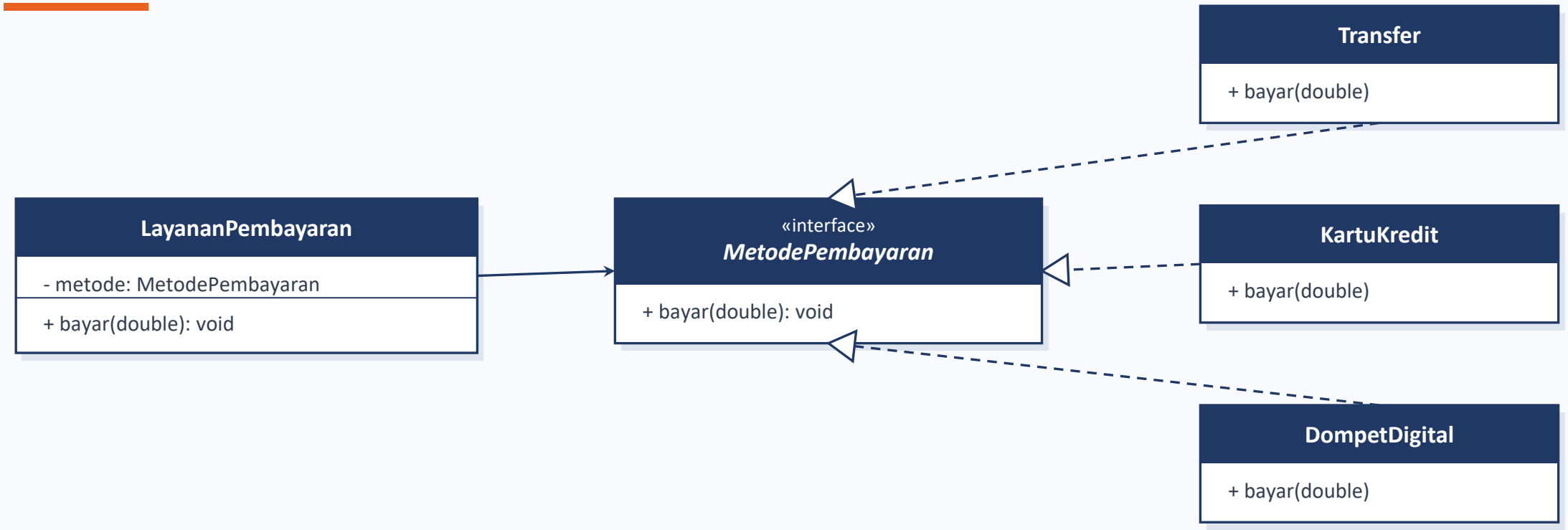
    LayananPembayaran(MetodePembayaran m) {
        this.metode = m;
    }
    void bayar(double n) {
        metode.bayar(n);   // tidak pernah berubah
    }
}
```

- **Pada kolom kanan, metode baru cukup menambah satu kelas yang memenuhi antarmuka.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Struktur Rancangan yang Memenuhi OCP

Kelas berbingkai kiri tertutup bagi modifikasi; kolom kanan terbuka bagi perluasan



- **Struktur ini kelak dikenal sebagai pola Strategy, yang dibahas pada pekan ke-11.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Panduan Praktis OCP

Dipakai pada latihan pekan ini dan pada Tugas 2 pekan depan

Daftar periksa

- Apakah menambah fitur baru menuntut kode lama diubah?
- Apakah variasi perilaku ditanam dalam if atau switch?
- Apakah kelas inti tetap utuh ketika fitur baru ditambahkan?
- Apakah ada instanceof yang menentukan perilaku?

Resep perbaikan

- Kenali bagian kode yang paling sering berubah karena kebutuhan.
- Ekstrak bagian itu menjadi antarmuka.
- Pindahkan tiap variasi ke kelas implementasi tersendiri.
- Serahkan pemilihan implementasi kepada pihak perakit atau factory.

© Fakultas Ilmu Komputer, Universitas Brawijaya

04

MATERI KE-4

Liskov Substitution Principle

Bagian 4 dari 4

© Fakultas Ilmu Komputer, Universitas Brawijaya

Substitusi Tanpa Kejutan

LSP adalah pertanyaan ketiga dari lima pertanyaan penguji pada pekan ke-2

Perumusan

- "Subtypes must be substitutable for their base types." (Liskov, 1987)
- **Makna.** Di mana pun tipe dasar dipakai, turunannya harus dapat menggantikannya.
- **Yang diuji.** Bukan sintaks, melainkan kontrak perilaku.

Mengapa penting

- **Polimorfisme bergantung padanya.** Bila kontrak dilanggar, polimorfisme berhenti bekerja.
- **Menjaga makna pewarisan.** Pewarisan bukan sekadar cara memakai ulang kode.
- **Menjaga OCP.** Klien yang harus memeriksa jenis objek tidak lagi tertutup bagi modifikasi.

Intinya: pelanggaran LSP hampir selalu berujung pada percabangan instanceof di sisi klien.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Gejala Pelanggaran LSP

Empat tanda yang biasanya muncul bersama-sama

1 Metode dikosongkan

Turunan membiarkan metode induk tanpa isi karena tidak relevan baginya.

2 Pengecualian dilempar

Turunan melempar UnsupportedOperationException pada metode yang seharusnya sah.

3 Aturan induk diubah

Turunan memperketat prasyarat atau memperlonggar jaminan hasil.

4 Klien memeriksa jenis

Muncul if (obj instanceof Turunan) di sisi pemakai.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kasus Klasik: Persegi sebagai Turunan Persegi Panjang

Persegi memang “adalah sebuah” persegi panjang secara matematis — tetapi kontraknya berbeda

PersegiPanjang.java

```
class PersegiPanjang {
    protected int lebar, tinggi;
    void setLebar(int l)  { this.lebar = l; }
    void setTinggi(int t) { this.tinggi = t; }
    int luas() { return lebar * tinggi; }
}

class Persegi extends PersegiPanjang {          // secara sintaks
sah
    @Override void setLebar(int l)  { lebar = tinggi = l; }
    @Override void setTinggi(int t) { lebar = tinggi = t; }
}
```

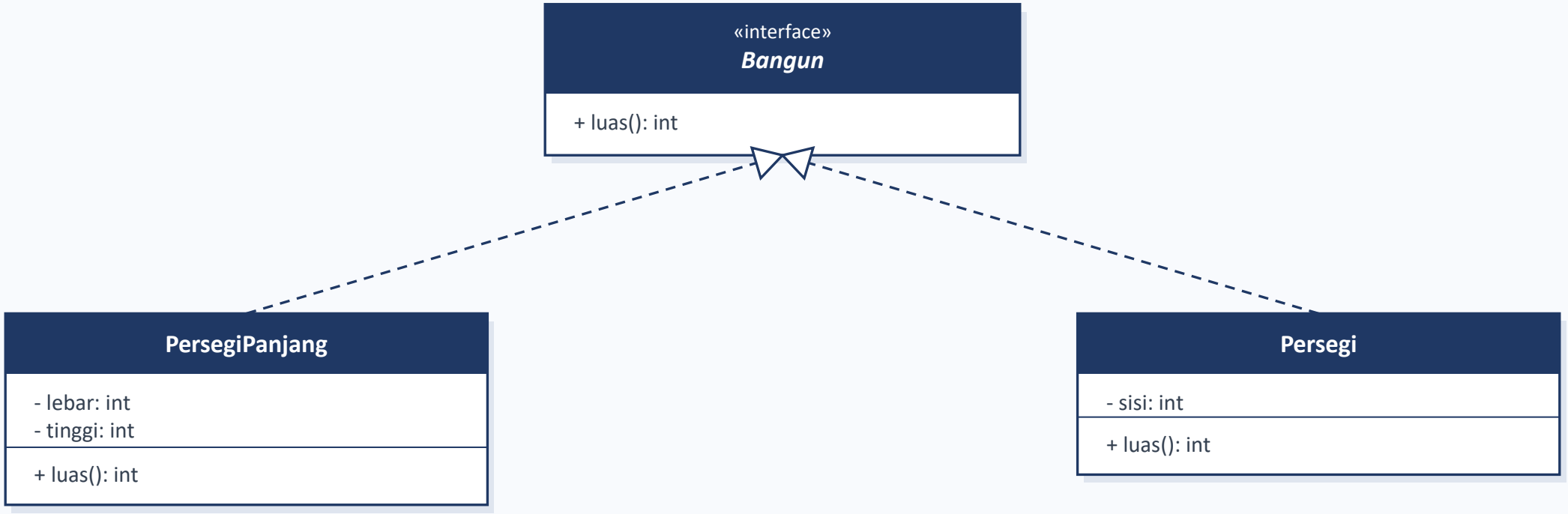
```
// Kontrak yang diandaikan klien: lebar dan tinggi saling bebas.
void uji(PersegiPanjang p) {
    p.setLebar(5);
    p.setTinggi(4);
    assert p.luas() == 20;      // gagal bila p bertipe nyata
Persegi
}
```

Yang dilanggar bukan tipe, melainkan janji perilaku yang diandaikan kode klien.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Perbaikan: Hierarki Disusun Ulang

Keduanya memenuhi kontrak yang sama tanpa saling mewarisi



- **Tidak ada janji yang dilanggar karena tidak ada janji tentang lebar dan tinggi yang saling bebas.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Dua Kasus Nyata di Lingkungan Akademik

Perhatikan bahwa perbaikan keduanya sama: mempersempit kontrak

Presensi mahasiswa

- **Melanggar.** PengenalanWajah mewarisi MetodePresensi tetapi membiarkan pindaiKartu() kosong.
- **Gejalanya.** Metode kosong dan pemeriksaan jenis di sisi pemanggil.
- **Perbaikan.** Persempit kontrak menjadi verifikasi(Mahasiswa) yang berlaku bagi semua metode.

Pembayaran uang kuliah

- **Melanggar.** PembayaranDompetDigital mewarisi PembayaranBank yang mewajibkan otorisasiKartu().
- **Gejalanya.** Turunan melempar pengecualian pada metode yang tidak relevan baginya.
- **Perbaikan.** Gunakan antarmuka MetodePembayaran dengan satu operasi bayar().

Intinya: mempersempit kontrak adalah Interface Segregation Principle, yang dibahas pekan depan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Ringkasan SRP, OCP, dan LSP

Tiga prinsip pertama, siap dipakai pada latihan dan Tugas 2

Prinsip	Pertanyaan penguji	Gejala pelanggaran	Arah perbaikan
SRP	Berapa aktor yang dilayani kelas ini?	Satu kelas berubah karena banyak permintaan berbeda	Pecah kelas menurut aktor
OCP	Apakah fitur baru menuntut kode lama diubah?	Percabangan panjang dan instanceof	Ekstrak antarmuka, pindahkan variasi
LSP	Dapatkah turunan menggantikan induk?	Metode kosong dan pengecualian tak terduga	Susun ulang hierarki, sempitkan kontrak

© Fakultas Ilmu Komputer, Universitas Brawijaya

Latihan di Kelas

Lima belas menit berpasangan, lalu dibahas bersama

1 Temukan pelanggarannya

Diberikan tiga potongan kode pendek.

Tugas: sebutkan prinsip yang dilanggar beserta baris penyebabnya.

2 Sebutkan aktornya

Untuk kasus SRP, daftarkan aktor tiap metode.

Tugas: tentukan berapa kelas yang seharusnya ada.

3 Usulkan perbaikan

Tuliskan antarmuka dan kelas hasil pemisahan.

Tugas: buat sketsa class diagram sederhananya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Simpulan

- SOLID adalah lima prinsip yang menjadikan mutu rancangan dapat dinilai dengan alasan yang terperiksa.
- SRP menuntut satu kelas hanya melayani satu aktor, sehingga hanya memiliki satu alasan untuk berubah.
- OCP menuntut perilaku baru dapat ditambahkan tanpa mengubah kode yang sudah stabil, melalui abstraksi dan polimorfisme.
- LSP menuntut turunan dapat menggantikan induknya tanpa melanggar janji perilaku yang diandaikan klien.
- Ketiga prinsip ini saling menopang: pelanggaran LSP merusak OCP, dan pelanggaran SRP menyulitkan keduanya.

Pekan depan: prinsip SOLID bagian kedua — Interface Segregation dan Dependency Inversion, serta keterkaitan kelima prinsip.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pustaka

Gaya pengacuan: Harvard-Anglia

- Martin, R.C., 2002. Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River, NJ: Prentice Hall, Bab 8–10.
- Refactoring Guru, 2024. Design Patterns. [daring] Tersedia pada: <https://refactoring.guru/design-patterns> [Diakses 5 Juli 2024].
- Liskov, B., 1987. Data Abstraction and Hierarchy. ACM SIGPLAN Notices, 23(5), hlm. 17–34.
- Meyer, B., 1988. Object-Oriented Software Construction. New York: Prentice Hall.

© Fakultas Ilmu Komputer, Universitas Brawijaya